Trabajo practico III



SELENIUM

Profesor: Juan Arce

Alumno: Genaro Meynet

SELENIUM

Selenium es un proyecto que engloba una serie de herramientas y bibliotecas que permiten y facilitan la automatización de los navegadores web. Se usa ampliamente para probar aplicaciones web, automatizar tareas repetitivas y raspar sitios web.

El núcleo de Selenium es WebDriver, una interfaz para escribir conjuntos de instrucciones que se pueden ejecutar indistintamente en varios navegadores. Una vez instalado todo, con solo unas pocas líneas de código podrás acceder a un navegador.

Selenium permite interactuar con páginas web mediante programación. Puedes navegar por las páginas, pulsar botones, rellenar formularios y extraer datos. Esto hace de Selenium una opción ideal para tareas que suponen interacciones repetitivas con interfaces web. Selenium se compone de varios componentes:

1. Selenium WebDriver: es el componente principal de Selenium, que proporciona API para interactuar con los navegadores web. Es compatible con varios navegadores, incluidos Chrome, Firefox, Safari y Edge. Es el que se encarga de navegar como si lo hiciera un usuario
2. Selenium IDE: es un entorno de desarrollo integrado para los scripts de Selenium, que permite grabar y reproducir las interacciones de los usuarios con las páginas web.
3. Selenium Grid: es una herramienta que permite ejecutar pruebas de Selenium en varios equipos y navegadores simultáneamente, lo que facilita la ejecución de pruebas en paralelo.

Los principales usos que utilizarse Selenium son:

- Hacer pruebas funcionales comprobando si los botones, formularios, menús, enlaces de una pagina web funcionan bien al simular a un usuario real
- Ejecutar pruebas de forma automática cada vez que se actualiza o modifica el código de una aplicación para asegurar que no se haya roto nada
- Verificar que un sitio web se vea y funcione de la misma manera en varios navegadores
- Automatización de tareas repetitivas como rellenar formularios extensos de forma masiva, descargar reportes periódicos o mover datos entre plataformas web que tienen una API abierta

Selenium no solo se usa para realizar pruebas, sino también para hacer Scraping a sitios web. Al automatizar las acciones del navegador, Selenium se puede usar para extraer datos de las páginas web de forma dinámica. Resulta útil para raspar sitios web con interacciones complejas de JavaScript que son difíciles de gestionar con las herramientas de raspado tradicionales, es decir, podemos simular la interacción de un usuario, pero de forma automática

Automatización de compras con Selenium

aplicación de consola desarrollada en C# que utiliza **Selenium WebDriver** para automatizar un proceso de compra en el sitio de pruebas **SauceDemo**.

El sistema permite al usuario seleccionar un producto, establecer un presupuesto máximo y, mediante Selenium, consultar los productos disponibles en la página y determinar si el producto seleccionado se encuentra dentro del presupuesto.

Si el producto cumple con el presupuesto, el sistema lo agrega al carrito y continúa automáticamente con el proceso de checkout.

Funcionamiento

El flujo principal del sistema es el siguiente:

1. Se configuran las credenciales de acceso al sitio.
2. Se inicia Google Chrome mediante Selenium WebDriver.
3. El sistema accede a SauceDemo e inicia sesión automáticamente.
4. El usuario selecciona uno de los productos disponibles.
5. El usuario ingresa un presupuesto máximo.
6. Selenium obtiene de la página el nombre y precio del producto seleccionado.
7. El sistema compara el precio del producto con el presupuesto ingresado.
8. Si el producto supera el presupuesto, se informa al usuario y se permite seleccionar otro producto.
9. Si el producto se encuentra dentro del presupuesto, Selenium lo agrega al carrito.
10. Se accede al carrito y se inicia el proceso de checkout.
11. El usuario ingresa sus datos personales y código postal.
12. Selenium completa el proceso de compra.

13. Finalmente, el sistema muestra un resumen de la compra realizada.

Estructura del proyecto

El proyecto está compuesto por tres clases principales:

Program.cs

Contiene el flujo principal de la aplicación. Se encarga de interactuar con el usuario, controlar las decisiones del programa y coordinar las acciones realizadas mediante Selenium.

Configuracion.cs

Contiene los datos utilizados durante la ejecución, como las credenciales, el producto seleccionado, el presupuesto máximo y los datos necesarios para el checkout.

También contiene la lógica para asociar la opción seleccionada por el usuario con el producto correspondiente.

Navegador.cs

Centraliza las operaciones realizadas sobre Google Chrome mediante Selenium WebDriver.

Entre sus funciones se encuentran:

- Abrir una URL.
- Buscar elementos por ID.
- Buscar elementos por clase.
- Escribir información en campos.
- Realizar clics.
- Obtener información del navegador.

Además, configura Chrome para reducir las interrupciones del navegador durante la automatización.

Tecnologías utilizadas

- C#
- .NET
- Selenium WebDriver
- Google Chrome
- SauceDemo

Manejo de errores

El sistema utiliza validaciones de entrada para evitar datos incorrectos, por ejemplo:

- Opciones de productos fuera del rango permitido.
- Presupuestos que no sean numéricos.
- Datos personales vacíos.
- Códigos postales inválidos.

También cuenta con manejo de excepciones mediante try-catch para evitar que un error de Selenium cierre el programa sin informar al usuario.

En caso de que Chrome muestre una alerta de seguridad que interfiera con la automatización, el sistema informa al usuario para que pueda cerrarla manualmente y continuar con la ejecución.

Objetivo del proyecto

El objetivo principal es demostrar el uso de **automatización web con Selenium WebDriver**, combinando la interacción con elementos HTML, extracción de información de una página web, toma de decisiones mediante lógica de programación y automatización de un proceso completo de compra.

Notas finales

El programa no fue tan fácil de afrontar como me pareció al principio. Estuve bastante tiempo tratando de averiguar de dónde venía una excepción que lanzaba el programa y no encontraba el motivo.

Después de probar y revisar varias cosas, descubrí que el problema era que Chrome mostraba un cartel de seguridad relacionado con la contraseña. Esto ocurre porque la contraseña que utilizamos es una contraseña de prueba de SauceDemo y Chrome la reconoce como una contraseña que apareció en una brecha de seguridad.

Probé diferentes alternativas para evitar que el cartel apareciera, pero no encontré una solución que funcionara de forma confiable. Por eso, finalmente decidí manejar la situación mediante una excepción: si el problema ocurre, el programa informa al usuario que debe cerrar manualmente el cartel de seguridad para poder continuar.

Aunque fue una dificultad que me llevó bastante tiempo resolver, también me sirvió para entender que en una automatización no todo depende del código que uno escribe, sino que también pueden intervenir factores externos, como el propio navegador.

obviamente, el programa podría seguir mejorándose y agregándole más funcionalidades, como una validación más completa de los datos o un manejo más específico de algunos errores. Sin embargo, considero que para el objetivo del trabajo funciona correctamente y sirve como un buen ejemplo de cómo se puede utilizar Selenium para automatizar acciones en una página web.

FUENTES

- Brightdata. ¿Qué es Selenium? https://brightdata.es/faqs/selenium/what-is-selenium

- WebDriver. https://www.w3.org/TR/webdriver/

- Selenium. El proyecto de automatización de navegadores con Selenium https://www-selenium-dev.translate.goog/documentation/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc

- Icaro. Tester Selenium https://icaro.org.ar/blog/tester-selenium/